

Neurolabs iOS SDK Integration Guide

Technical Integration Specification

Document ID: NLB-IOS-INTEGRATION-GUIDE

Version: v1.1.16

Date: 2026-03-17

Status: Partner integration reference

Owner: Neurolabs

1. Scope

This guide is for partner iOS apps integrating 'NeurolabsSDK' through Swift Package Manager.

2. Changes Since v1.1.3

- ? No iOS SDK public API integration changes were introduced after 'v1.1.3' in this repository.
- ? Existing integration steps remain valid.

3. Requirements

- ? iOS 15+
- ? Xcode 15+
- ? Swift 5.9+

4. Install (SPM)

Add package dependency (mobile-dist or direct distribution manifest) and link 'NeurolabsSDK'.

5. SDK Initialization + Warmup

```
import NeurolabsSDK

final class CaptureCoordinator {
    let sdk = NeurolabsSDKCore(configuration: .init(
        apiKey: "<API_KEY>",
        apiBaseUrl: URL(string: "https://api.neurolabs.ai")!,
        demoIRTaskUUID: "<DEFAULT_TASK_UUID>",
        debugLogging: true
    ))

    func prepare() {
        // SDK starts image pipeline preparation during init.
        // Optional: observe loading progress through delegate.
    }
}
```

6. Queue Management

```

Task {
  await sdk.setAutoSyncEnabled(true)
  await sdk.setWifiOnlyUploadsEnabled(false)
  await sdk.setDeletePhotosOnUploadSuccess(true)
  await sdk.setMaxQueueSize(200)
  await sdk.setUploadRetryCount(5)

  let status = await sdk.getQueueStatus()
  print("pending=\\(status.pendingCount) failed=\\(status.failedCount)")

  await sdk.flushQueue()
  await sdk.retryFailedUploads()
}

```

7. Open Custom Camera

```

import UIKit
import NeurolabsSDK

var capture = NLCaptureConfiguration()
capture.confidenceThreshold = 0.25
capture.iouThreshold = 0.45
capture.maxCaptures = 1
capture.guidanceMode = .strict
capture.showDetectionOverlays = false
capture.showCapturedRegions = false

let config = NLCameraConfiguration(
  capture: capture,
  enableValidation: true,
  showAlignmentGuidance: true,
  enableARDetections: false,
  liveQualityEnabled: true,
  liveQualityFPS: 6,
  showCapturePreview: true,
  requiredCaptureCount: 1,
  autoCloseAfterCapture: true,
  sendDetectionsMetadata: true,
  enablePreviewCropping: true
)

let handlers = NLCustomCameraHandlers(
  onSave: { capture in
    // Custom post-processing hook for each accepted capture
    // return .performDefault -> SDK also enqueues upload
    // return .skipDefault -> app takes full ownership
    return .performDefault
  },
  onSaveAll: { captures in
    // Batch hook (multi-capture flow)
    return .performDefault
  }
)

sdk.openCustomCameraUI(
  configuration: config,
  handlers: handlers,
  onClose: {
    print("Camera closed")
  }
)

```

Recommended capture payload notes:

- ? 'liveQualityEnabled: true' is the key for pill/rotation/warning/error guidance behavior in the custom camera.
- ? Keep shelf capture mode with strict guidance and overlays disabled for the custom-camera guidance UI path.
- ? Use 'autoCloseAfterCapture: true' with 'requiredCaptureCount: 1' for the current parity flow.

8. Post-Processing Hooks

- ? 'onSave' receives each approved capture.
- ? 'onSaveAll' receives final batch in multi-capture mode.
- ? 'onUploadRequested' exists but default queueing is done in 'onSave' / 'onSaveAll' path.

```
let handlers = NLCustomCameraHandlers(  
  onSave: { capture in  
    MyPostProcessor.shared.enqueue(capture)  
    return .skipDefault  
  }  
)
```

9. Delegate/Event Callbacks

```
final class SDKDelegate: NeurolabsSDKDelegate {  
  func neurolabsSDK(_ sdk: NeurolabsSDKCore, didProduceCaptureResult result:  
    NLNativeCaptureResult, for captureId: String, sessionId: String) {}  
  func neurolabsSDK(_ sdk: NeurolabsSDKCore, didEncounterError error: NSError, sessionId:  
    String?, messageId: String?) {}  
  func neurolabsSDK(_ sdk: NeurolabsSDKCore, didChangeQueueStatus status: NLQueueStatus) {}  
  func neurolabsSDK(_ sdk: NeurolabsSDKCore, didUpdateLoadingProgress progress:  
    NLLoadingProgress) {}  
}  
  
sdk.delegate = SDKDelegate()
```

10. Notes

- ? Per-session routing is supported via native capture config task UUID overrides.
- ? Ensure 'NSCameraUsageDescription' is present in app 'Info.plist'.